

Rapport TP1 – Génie Logiciel

ENSA Béni Mellal

Module : Génie Logiciel

TP : Programmation Orientée Objet en Java

Réalise par : Fatima Ezzahra IGHIL

Introduction

Ce TP a pour objectif de comprendre et d'appliquer les concepts fondamentaux de la Programmation Orientée Objet (POO) en Java, notamment :

- L'encapsulation
- L'héritage
- L'abstraction
- Le polymorphisme

Trois exercices ont été réalisés afin de manipuler ces concepts dans des contextes différents.

Exercice 1.1 : Gestion des animaux

Objectif

Modéliser un système de gestion d'animaux en utilisant les concepts POO.

Code réalisé

Le code comprend :

- Une classe abstraite `Animal`
 - Deux classes dérivées `Mammifere` et `Oiseau`
 - Une méthode spécifique `voler()` pour les oiseaux
 - Une classe principale `Zoo`
-

Réponses théoriques

1. Encapsulation

L'encapsulation consiste à protéger les données d'une classe en les rendant privées et en contrôlant leur accès via des méthodes.

Dans la classe `Animal`, l'attribut `nom` est privé et accessible uniquement via la méthode `getNom()`.

2. Héritage

L'héritage permet de créer une classe à partir d'une autre.

Dans cet exercice :

- `Mammifere` hérite de `Animal`

- `Oiseau` hérite de `Animal`

3. Abstraction

L'abstraction consiste à définir une classe générale sans implémenter tous les détails.

La classe `Animal` est abstraite et contient une méthode abstraite `faireDuBruit()`.

4. Polymorphisme

Le polymorphisme permet d'utiliser une même méthode avec différents comportements.

Dans `main()` :

- `tigre.faireDuBruit()` → grogne
- `perroquet.faireDuBruit()` → chante

5. Méthode voler()

Une méthode `voler()` a été ajoutée dans la classe `Oiseau` pour afficher :

```
| "Perroquet vole."
```

6. Modification de main()

Un cast a été utilisé pour appeler la méthode `voler()` :

```
Oiseau oiseau = (Oiseau) perroquet;  
oiseau.voler();
```

7. Mots-clés utilisés

- Héritage : `extends`
- Abstraction : `abstract`
- Redéfinition : `@Override`

✓ Résultat obtenu

Le programme affiche :

- Tigre grogne
- Perroquet chante
- Perroquet vole

◆ Exercice 1.2 : Gestion des véhicules

✓ Objectif

Créer un système de gestion de véhicules en utilisant l'héritage et le polymorphisme.

✓ Code réalisé

Le code comprend :

- Une classe abstraite `Vehicule`
 - Trois classes dérivées : `Voiture`, `Camion`, `Moto`
 - Une méthode concrète `demarrer()`
-

✓ Réponses théoriques

1. Encapsulation

Les attributs (`marque`, `modele`, `annee`) sont privés et accessibles via des getters.

2. Héritage

Les classes `Voiture`, `Camion` et `Moto` héritent de `Vehicule`.

3. Polymorphisme

La méthode `afficherDetails()` est redéfinie dans chaque classe.

4. Abstraction

La classe `Vehicule` est abstraite car elle contient une méthode abstraite `afficherDetails()`.

5. Méthode `demarrer()`

Une méthode `demarrer()` a été ajoutée et appelée dans chaque classe.

6. Classe `Moto`

Une classe `Moto` a été créée avec un attribut `typeDeGuidon`.

7. Mots-clés utilisés

- Héritage : `extends`
 - Redéfinition : `@Override`
 - Constructeur parent : `super`
-

✓ Résultat obtenu

Le programme affiche les informations de chaque véhicule avec le message :

| "Le véhicule démarre."

◆ Exercice 1.3 : Gestion des comptes bancaires

✓ Objectif

Créer un système de gestion de comptes bancaires.

✓ Code réalisé

Le code comprend :

- Une classe `CompteBancaire`

- Deux classes dérivées : `CompteCourant` et `CompteEpargne`
 - Des méthodes de gestion : dépôt, retrait, calcul d'intérêts
-

✓ Fonctionnalités

✓ Dépôt

Ajoute un montant au solde

✓ Retrait

Vérifie si le solde est suffisant

✓ Intérêts

Ajoute un pourcentage au solde (`CompteEpargne`)

✓ Résultat obtenu

Le programme affiche :

- Solde du compte courant
 - Solde du compte épargne après intérêts
-

◆ Conclusion

Ce TP nous a permis de :

- Comprendre les concepts fondamentaux de la POO
- Appliquer l'héritage et le polymorphisme
- Structurer un programme Java de manière claire

Nous avons également appris à :

- Organiser le code en classes
- Réutiliser du code grâce à l'héritage
- Adapter le comportement des objets